开源软件基础

数据库 sqlite3

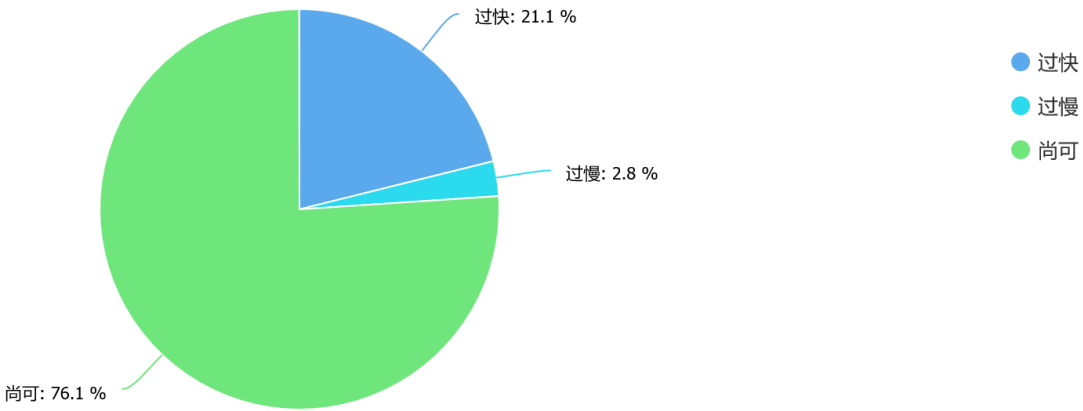
Zhilei Ren

OSCAR Team, School of Software, Dalian University of Technology

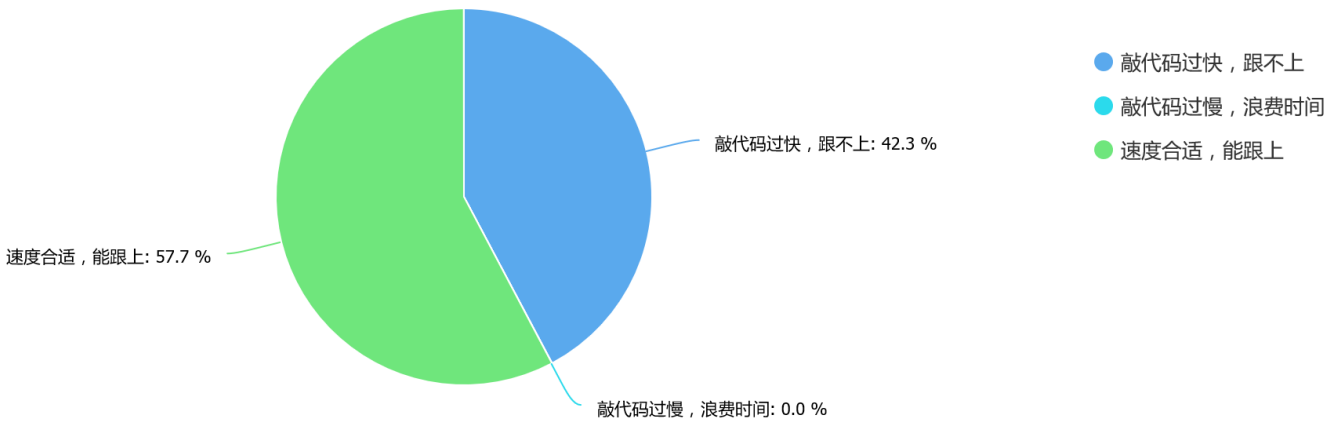
October 19, 2020

Outline

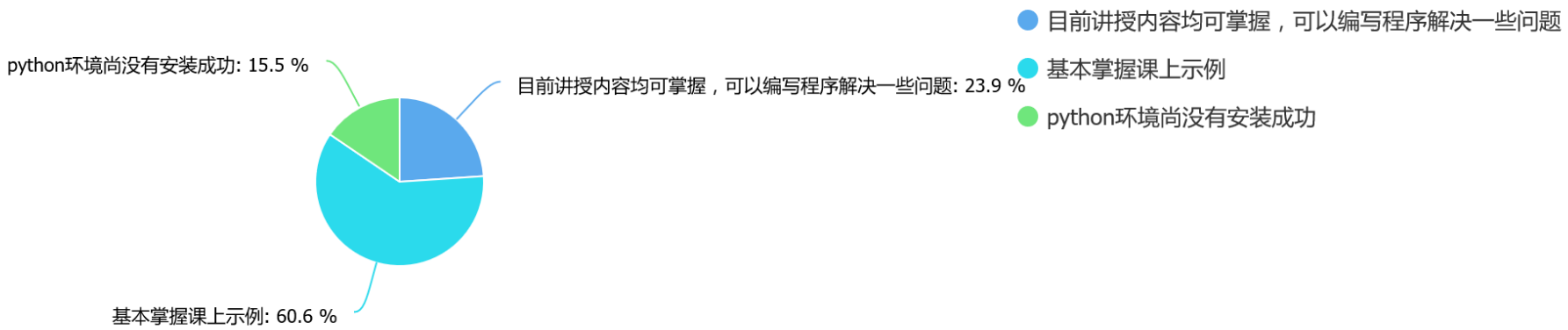
课堂进度如何



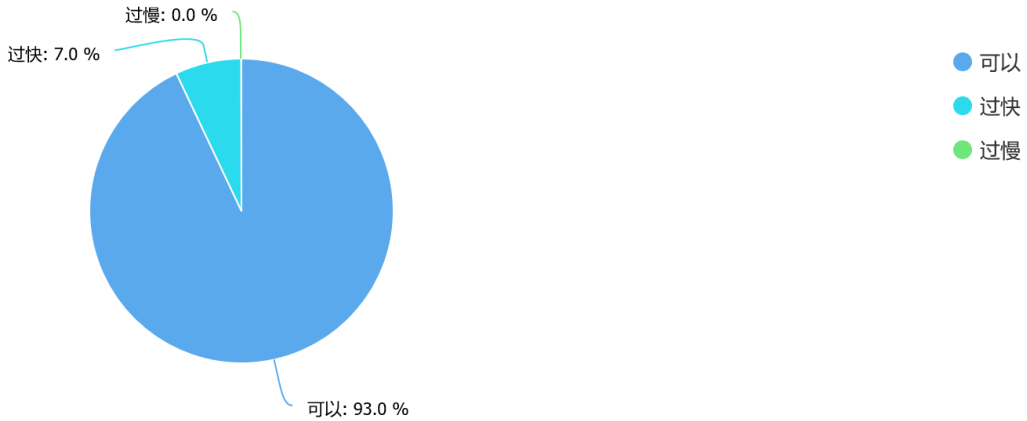
课堂现场编程示例效果如何



python 知识掌握程度



上课语速如何



对于课程有什么其他意见或建议

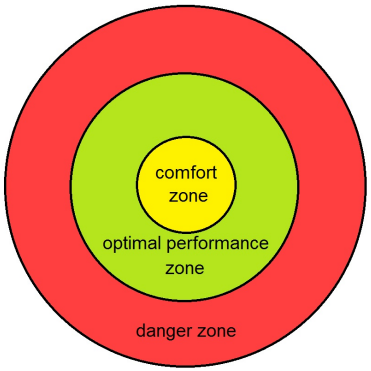
典型反馈

- 多讲些故事吧
- 老师可以推荐几本学 Python 的书吗
- 少喝一点可乐比较好
- 有时敲代码太快来不及抄就过去了
- 希望能讲讲 linux
- 便于课下练习，适当加些课后练习

今天就能用的小知识

舒适区

舒适区（英语：Comfort zone）或舒适圈，指的是一个人所处的一种环境的状态，和习惯的行动，人会在这种安乐窝的状态中感到舒适并且缺乏危机感。
走出舒适区会增加人的焦虑程度，从而产生应激反应，其结果是提升对工作的专注程度。在这个区域中被称作最佳表现区。
但是罗伯特·耶基斯于 1907 年的报告中提到“焦虑可以改善工作表现，但是当超过某一最佳激励状态之后，工作表现就开始恶化”。[a]



[a] https://zh.wikipedia.org/wiki/

今天就能用的小知识

直到名扬天下你对我把头点
最基本的招练到烂熟就是杀手锏

– C-Block（野家拳）

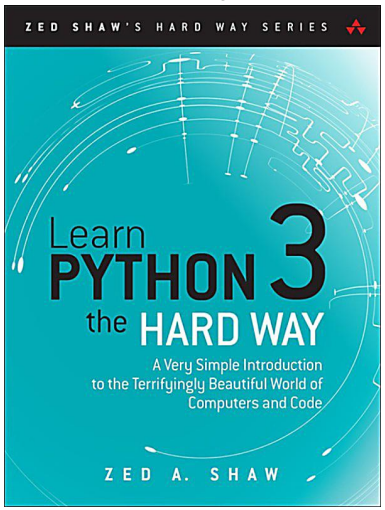
今天就能用的小知识

无他，但手熟尔

– 欧阳修（卖油翁）

书籍推荐

Learn Python 3 the hard way (笨办法学 Python)[1]



[1] https://learnpythonthehardway.org/python3/

Outline

文件打开与关闭

```
open(file, mode='r', buffering=-1, encoding=None,
     errors=None, newline=None,
     closefd=True, opener=None)^^I
```

标志	含义
'r'	读（默认）
'w'	写（覆盖）
'x'	排他性创建
'a'	写（追加）
'b'	二进制模式
't'	文本模式（默认）
'+'	更新（读写）
'U'	统一 newlines 模式（过时）

文件读写

```
handle = open("input.txt", "r")
content = handle.read()
handle.close()
#playing around with content
handle = open("output.txt", "w")
handle.write(content)
handle.close()
```

Outline

CSV

The so-called CSV (Comma Separated Values) format is the most common import and export format for spreadsheets and databases. CSV format was used for many years prior to attempts to describe the format in a standardized way in RFC 4180. The lack of a well-defined standard means that subtle differences often exist in the data produced and consumed by different applications. These differences can make it annoying to process CSV files from multiple sources.

CSV

读写方法

- 使用 reader() 和 writer() 构造读写对象
- 参数均为文件句柄
- reader 对象可以遍历（或转为 list）
- writer 对象用 writerows/writerow 写行

CSV

```
>>> import csv
>>> students = [['name', 'gender', 'age'],
... ['zhangsan', 'male', 13],
... ['lisi', 'female', 14]]
>>> handle = open('students.csv', 'w')
>>> writer = csv.writer(handle)
>>> writer.writerows(students)
>>> handle.close()
>>> handle = open('students.csv', 'r')
>>> reader = csv.reader(handle)
>>> for row in reader:
...     print(row)
```

Outline

sqlite3 — DB-API 2.0 interface for SQLite databases

SQLite is a C library that provides a lightweight disk-based database that doesn't require a separate server process and allows accessing the database using a nonstandard variant of the SQL query language. Some applications can use SQLite for internal data storage. It's also possible to prototype an application using SQLite and then port the code to a larger database such as PostgreSQL or Oracle.

sqlite3 — DB-API 2.0 interface for SQLite databases

The sqlite3 module was written by Gerhard Häring. It provides a SQL interface compliant with the DB-API 2.0 specification described by PEP 249.

sqlite3 — DB-API 2.0 interface for SQLite databases

To use the module, you must first create a Connection object that represents the database.
You can also supply the special name :memory: to create a database in RAM.

sqlite3 — DB-API 2.0 interface for SQLite databases

```
import sqlite3
conn = sqlite3.connect('example.db')
# or use :memory:
conn_memory = sqlite3.connect(':memory:')
```

sqlite3 — DB-API 2.0 interface for SQLite databases

Once you have a Connection, you can create a Cursor object and call its Cursor.execute method to perform SQL commands:

sqlite3 — DB-API 2.0 interface for SQLite databases

```
c = conn.cursor()

# Create table
c.execute('''CREATE TABLE stocks
             (date text, trans text, symbol text, qty real, price real)''')

# Insert a row of data
c.execute("INSERT INTO stocks VALUES
             ('2006-01-05','BUY','RHAT',100,35.14)")

# Save (commit) the changes
conn.commit()

# We can also close the connection if we are done with it.
# Just be sure any changes have been committed or they will be lost.
conn.close()
```

sqlite3 — DB-API 2.0 interface for SQLite databases

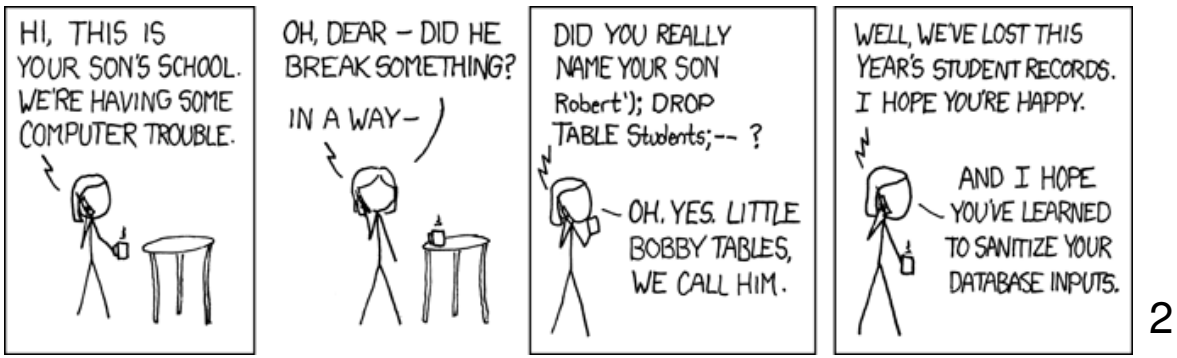
The data you've saved is persistent and is available in subsequent sessions:

sqlite3 — DB-API 2.0 interface for SQLite databases

```
import sqlite3
conn = sqlite3.connect('example.db')
c = conn.cursor()
```

sqlite3 — DB-API 2.0 interface for SQLite databases

Usually your SQL operations will need to use values from Python variables. You shouldn't assemble your query using Python's string operations because doing so is insecure; it makes your program vulnerable to an SQL injection attack.



2

[2] https://xkcd.com/327/

sqlite3 — DB-API 2.0 interface for SQLite databases

Instead, use the DB-API's parameter substitution. Put ? as a placeholder wherever you want to use a value, and then provide a tuple of values as the second argument to the cursor's Cursor.execute method. (Other database modules may use a different placeholder, such as %s or :1.) For example:

sqlite3 — DB-API 2.0 interface for SQLite databases

```
# Never do this -- insecure!
symbol = 'RHAT'
c.execute("SELECT * FROM stocks WHERE symbol = '%s'" % symbol)

# Do this instead
t = ('RHAT',)
c.execute('SELECT * FROM stocks WHERE symbol=?', t)
print(c.fetchone())

# Larger example that inserts many records at a time
purchases = [('2006-03-28', 'BUY', 'IBM', 1000, 45.00),
             ('2006-04-05', 'BUY', 'MSFT', 1000, 72.00),
             ('2006-04-06', 'SELL', 'IBM', 500, 53.00),
            ]
c.executemany('INSERT INTO stocks VALUES (?,?,?,?,?)', purchases)
```

Using shortcut methods

Using the nonstandard execute, executemany and executescript methods of the Connection object, your code can be written more concisely because you don't have to create the (often superfluous) Cursor objects explicitly.

Using shortcut methods

```
$ cat a.sql

create table news (id integer, score integer, title text, href text);
insert into news values (1, 8, "hello world", "http://oscar-lab.org");
insert into news values (2, 2, "hello charlie", "http://www.dlut.edu.cn");
$ python3
>>> import sqlite3
>>> conn = sqlite3.connect('a.db')
>>> c = conn.cursor()
>>> c.executescript(open('a.sql').read())
>>> list(c.execute('select * from news'))
[(1, 8, 'hello world', 'http://oscar-lab.org'),
 (2, 2, 'hello charlie', 'http://www.dlut.edu.cn')]
```

Row Objects

```
>>> conn.row_factory = sqlite3.Row
>>> c = conn.cursor()
>>> c.execute('select * from stocks')
<sqlite3.Cursor object at 0x7f4e7dd8fa80>
>>> r = c.fetchone()
>>> type(r)
<class 'sqlite3.Row'>
>>> tuple(r)
('2006-01-05', 'BUY', 'RHAT', 100.0, 35.14)
>>> len(r)
5
>>> r[2]
'RHAT'
```

sqlite3 — DB-API 2.0 interface for SQLite databases

To retrieve data after executing a SELECT statement, you can either treat the cursor as an iterator, call the cursor's Cursor.fetchone method to retrieve a single matching row, or call Cursor.fetchall to get a list of the matching rows.

sqlite3 — DB-API 2.0 interface for SQLite databases

```
>>> for row in c.execute('SELECT * FROM stocks ORDER BY price'):
        print(row)

('2006-01-05', 'BUY', 'RHAT', 100, 35.14)
('2006-03-28', 'BUY', 'IBM', 1000, 45.0)
('2006-04-06', 'SELL', 'IBM', 500, 53.0)
('2006-04-05', 'BUY', 'MSFT', 1000, 72.0)
```

类型对应

SQLite 默认支持如下类型: NULL, INTEGER, REAL, TEXT, BLOB.

Table 1: 与 Python 类型间对应关系

Python type	SQLite type
None	NULL
int	INTEGER
float	REAL
str	TEXT
bytes	BLOB

Outline

JSON

JSON

JSON(JavaScript Object Notation, JS 对象标记) 是一种轻量级的数据交换格式。它基于 ECMAScript (w3c 制定的 js 规范) 的一个子集，采用完全独立于编程语言的文本格式来存储和表示数据。简洁和清晰的层次结构使得 JSON 成为理想的数据交换语言。易于人阅读和编写，同时也易于机器解析和生成，并有效地提升网络传输效率。

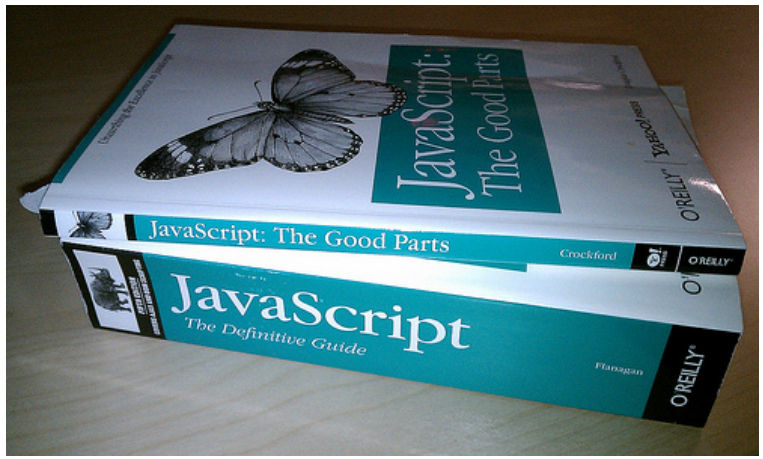


Figure 1: JavaScript: the definitive guide vs. the good parts

JavaScript

今天就能开始用的小知识

- JavaScript 学名 ECMAScript
- JavaScript 与 Java 的关系约等于济科和张继科的关系
- 相比于 Java，更接近 Lisp/Scheme
- 设计周期为 10 天[a][b][c][d]

[a] https://www.w3.org/community/webed/wiki/A_Short_History_of_JavaScript

[b] https://www.bilibili.com/video/av1623152/

[c] https://www.destroyallsoftware.com/talks/the-birth-and-death-of-javascript

[d] https://gist.github.com/zhangchn/29015a7f65ce11c94846

Example of JSON[3]

```
{
  "firstName": "John",
  "lastName": "Smith",
  "isAlive": true,
  "age": 25,
  "address": {
    "streetAddress": "21 2nd Street",
    "city": "New York",
    "state": "NY",
    "postalCode": "10021-3100"
  },
  "phoneNumbers": [
    {
      "type": "home",
      "number": "212 555-1234"
    },
    {
      "type": "office",
      "number": "646 555-4567"
    },
    {
      "type": "mobile",
      "number": "123 456-7890"
    }
  ],
  "children": [],
  "spouse": null
}
```

[3] https://en.wikipedia.org/wiki/JSON

JSON

```
>>> import json
>>> json.dumps(['foo', {'bar': ('baz', None, 1.0, 2)}])
'["foo", {"bar": ["baz", null, 1.0, 2]}]'
>>> print(json.dumps({"c": 0, "b": 0, "a": 0}, sort_keys=True))
{"a": 0, "b": 0, "c": 0}
>>> handle = open("output.json", "w")
>>> json.dump({'foo': 123, 'bar': 'buz'}, handle)
>>> handle.close()
```

JSON

```
>>> import json
>>> json.loads('["foo", {"bar":["baz", null, 1.0, 2]}]')
['foo', {'bar': ['baz', None, 1.0, 2]}]
>>> from io import StringIO
>>> handle = open('output.json', ``r'')

>>> b = json.load(handle)

>>> print(b)
{'foo': 123, 'bar': 'buz'}
```

Outline

Pickle

Pickle

The pickle module implements binary protocols for serializing and de-serializing a Python object structure. “Pickling” is the process whereby a Python object hierarchy is converted into a byte stream, and “unpickling” is the inverse operation, whereby a byte stream (from a binary file or bytes-like object) is converted back into an object hierarchy.

Pickle vs. JSON

Comparison with JSON

1. JSON is a text serialization format (it outputs unicode text, although most of the time it is then encoded to utf-8), while pickle is a binary serialization format;
2. JSON is human-readable, while pickle is not;
3. JSON is interoperable and widely used outside of the Python ecosystem, while pickle is Python-specific;
4. JSON, by default, can only represent a subset of the Python built-in types, and no custom classes; pickle can represent an extremely large number of Python types (many of them automatically, by clever usage of Python's introspection facilities; complex cases can be tackled by implementing specific object APIs);
5. Unlike pickle, deserializing untrusted JSON does not in itself

Example of Pickle

```
>>> import pickle
# Save a dictionary into a pickle file.

>>> favorite_color = {"lion": "yellow", "kitty": "red"}

>>> pickle.dump(favorite_color, open( "save.p", "wb"))

>>> favorite_color = pickle.load(open( "save.p", "rb"))

>>> serialized = pickle.dumps(favorite_color)

>>> deserialized = pickle.loads(serialized)
```